JD.com

Coding Agent Memory Post-Training II

Unlocking the Memory Potential of Pre-trained File Operations for Long-Horizon Tasks via On-Policy Distillation

Lirui Luo · Kelong Mao · Heming Xia · Rongqing Li · Xinwei Yang · Luyu Chen
Kieran Wong · Yudong Guo · Xinrui Wang · Jiayin Zhu · Simiu Gu · Sulong Xu · Cong Fang

JD.com

Project: liruiluo.github.io/agentmemorygym-v2 **Paper:** PDF

An interface teaches only what both models already speak.

Distillation asks two questions, not one

JD.com

Task-reward training

Can the student reach the behaviour by exploring?

One prior is enough

On-policy distillation

Does the student emit the action, and can the teacher score what it wrote?

Two priors must meet

Coverage and supervision are the two sides of the same design choice.

The interface has to satisfy both sides

JD.com

Student side

rollout

```
cat > notes.md
confirmed: velvet
```

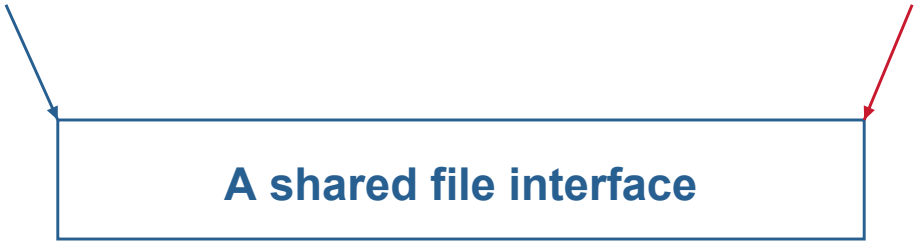
It must appear in the sample.

Teacher side

frozen teacher

```
p(note | context) = 0.91
```

It must be able to score the content.



One interface raises coverage and supervision at the same time.

The file interface is already familiar to both

JD.com

1/3

of the teacher's cost on compaction

21.6%

of actions are memory on files, against 7.8% and 5.9%

2.97x

gain over the best OPD baseline

Files cost the teacher a third of what compaction costs it, and the student a fifth. On the file interface the student writes and reads memory far more often than on the interfaces that need a protocol.

Nothing about the interface has to be taught before training starts.

CAMG-II: five environments, one memory substrate

JD.com

Shop	six sessions, one purchase chain
Coding	repository repair across the horizon
AutoResearch	experiment, read, revise
General	long office tasks over a tool surface
Visual	build, inspect the render, revise

Every environment keeps its native task and shares ordinary shell access with an episode-persistent workspace.

Distil on what the student actually writes

JD.com

Student samples → **Teacher scores every token** → **Learner follows the gap**

what the trainee wrote

```
page: css-only, no html body markup
plan: rewrite dist/index.html
next: rebuild and check
```

what the teacher returns

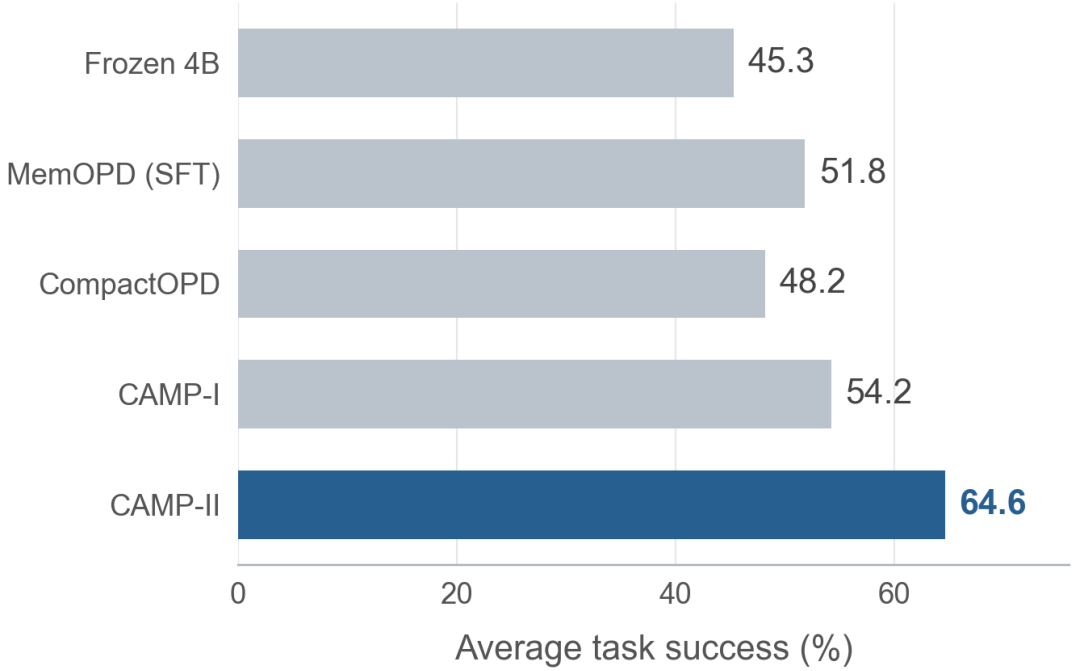
```
p(token | context) for every
token of the write
```

Rollouts keep arriving from published weights while the learner catches up, so an episode can enter a minibatch several weight publications after it was sampled.

Supervision reaches exactly the tokens the policy chose to write.

CAMP-II reaches 64.6% average task success

JD.com



64.6%

average over five domains

+10.4

points over CAMP-I

+16.4

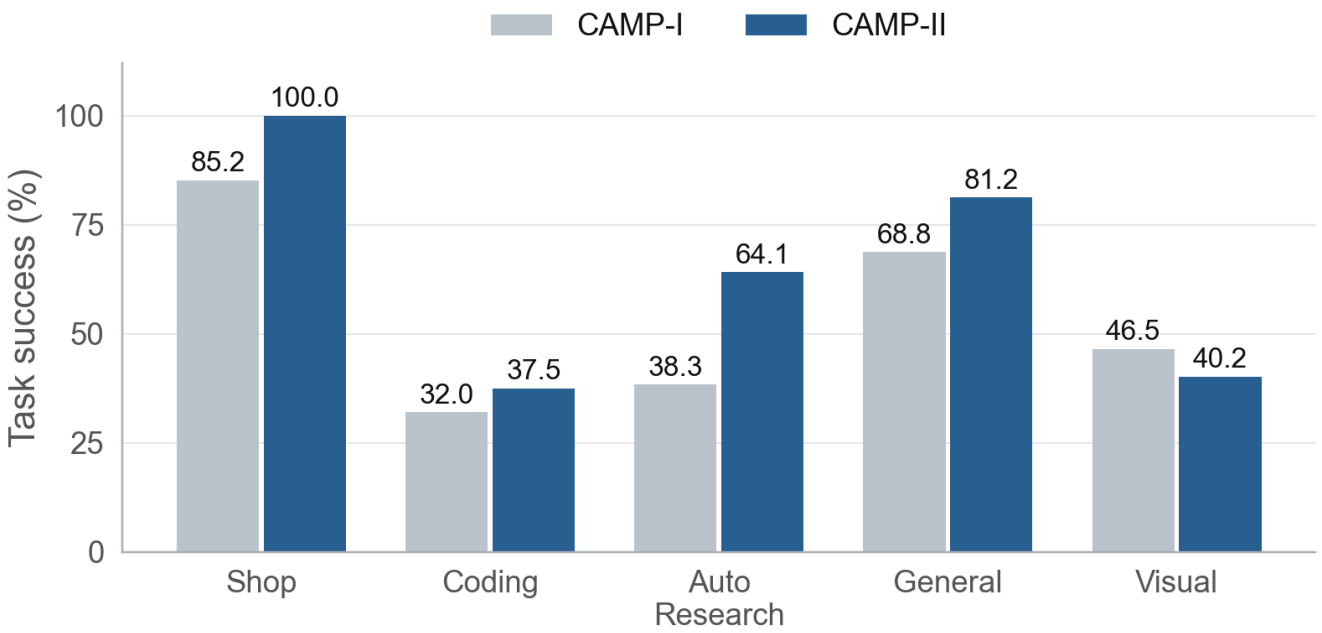
points over the frozen student

Same schedule, same budget - a different interface.

Table 1 · five held-out panels, 128 tasks each

The gain lands where the interface carries the most

JD.com



+25.8
AutoResearch

+12.4
General

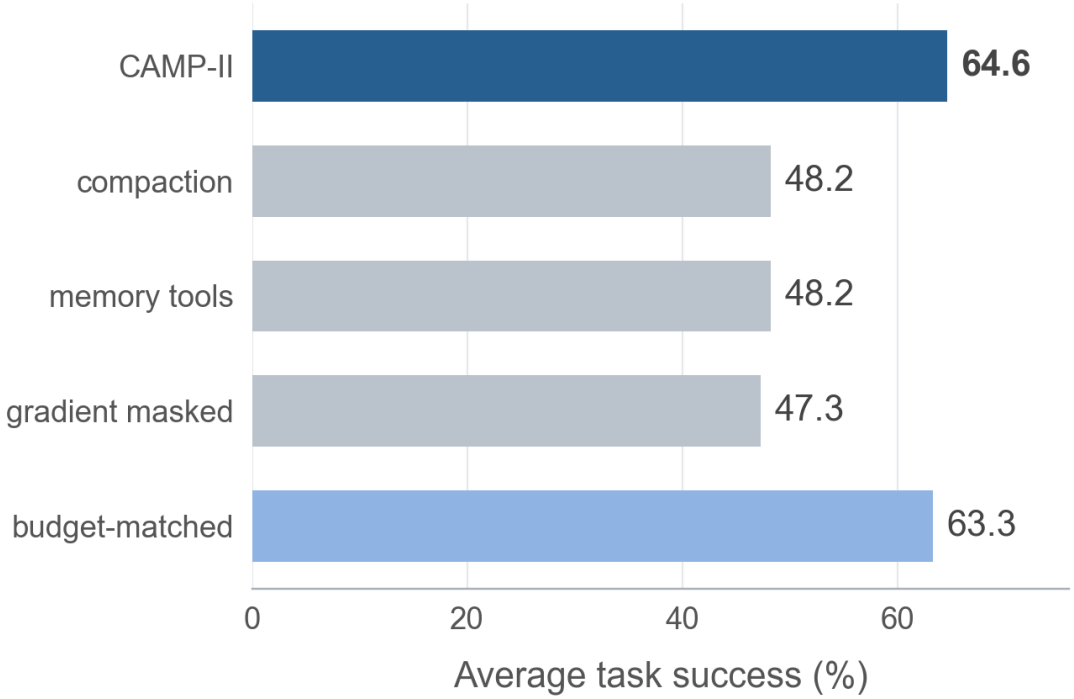
-6.3
Visual

Shop saturates and Visual stays bounded by the judge; the long horizons open up.

Table 1

Every variant that touches memory supervision loses the gain

JD.com



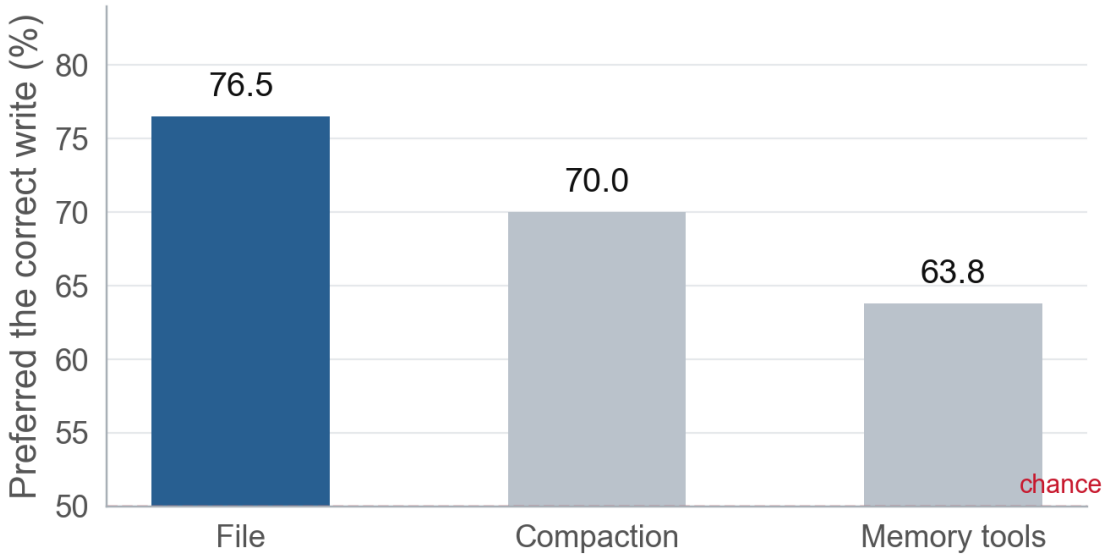
Swapping the interface for compaction or for the dedicated memory tools costs 16 to 17 points. Masking the loss on the tokens the policy writes costs 17. Masking the same number of tokens at random costs 1.3.

The gain is the supervision on memory content, **not the budget.**

Figure 6 · Section 6.2

The teacher prefers the write that keeps the task

JD.com



Two hundred note pairs per interface, each scored in both orders under the same visible context. The teacher ranks the write that carries the prefix's own task above a foreign one.

0.918

on Shop, with three distinct values

A familiar interface gives the teacher something to compare.

What the note says decides what the next context can do

JD.com

Task: Build a fitness-product showcase

A static gallery with product cards, collections, comparisons and customer reviews.

(a) Write task memory

`CONTR.txt` Turn 32

Problem

"a malformed CSS-only file"

"NO HTML body markup"

Plan

"Complete rewrite of dist/index.html with all sections"

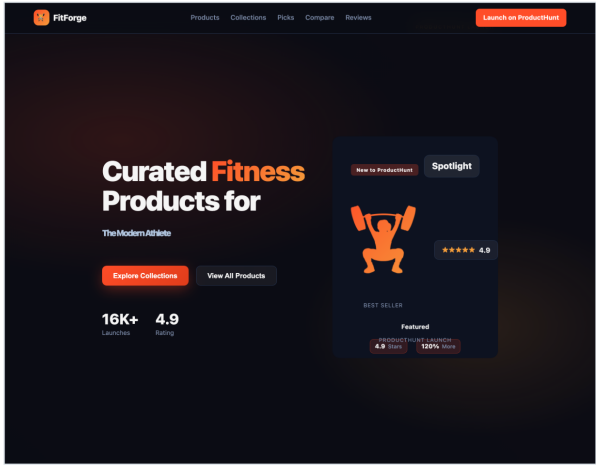
"Featured products" · "Trending collections"
"Editor picks" · "Comparison cards"

Context replacement · turn 33

(b) Retrieve task memory

Turn 35	`ls -la`
Turn 36	`cat CONTR.txt`

(c) Continue development Turns 37–54



Change the saved content and the episode that follows it changes with it.

Figure 5 · intervention on the stored note

The result does not rest on one draw

JD.com

64.9
mean over five seeds

0.5
standard deviation, points

0.9
largest domain deviation

seed / average		
1 64.6	2 65.5	3 64.4
4 65.0	5 65.2	

Shop stays at 100.0 in every seed. Coding, AutoResearch, General and Visual each move by about a point, and the ordering of the domains is stable.

The comparison in the paper holds across training runs.

Appendix C.3 · Table 8

Trained memory carries outside the gym

JD.com

Policy	Terminal-Bench 2.1	SWE-Bench Pro V2	Average
Qwen3.5-4B	23.1	11.2	17.1
Qwen3.5-9B	23.1	23.1	23.1
Qwen3.5-35B-A3B	33.8	27.4	30.6
CAMP-II-4B	**30.0**	**18.1**	**24.1**
CAMP-II-9B	**38.0**	**30.1**	**34.1**

Both trained policies improve on the weights they started from. The 9B policy averages above a 35B model at a quarter of its parameters.

Table 2 · same suites, decoding budgets and graders across rows

Why the interface decides the cost

JD.com

Rare-action regime of the mean-field cost model:

$$N = 1 / (2 \lambda Q_0 u_0)$$

Q_0 how rarely the student emits the action

u_0 how far the student starts from the teacher

The exact mean-field integral has two limits. Rare action gives the form above; a policy already close to the teacher loses u_0 entirely and the cost falls to $\log(1/\rho)$ over $2\lambda Q(1-Q)$. The law is a limit, not a universal law, and the paper says which regime it applies to.

An interface moves Q_0 and u_0 at the same time.

What this paper establishes

JD.com

A shared interface prior sets what memory distillation can teach.

CAMP-II reaches 64.6% average over five task worlds without a memory-specific teacher.

The behavior survives five seeds and transfers to two external suites.

Code and weights: withheld during review.

liruiluo.github.io/agentmemorygym-v2